

Project Report: Java Chess Game

Date: June 25, 2025

Project Title: Java Chess Application with AI

1. Introduction

This report details a comprehensive chess application developed in Java. The project's primary objective was to create a fully functional and user-friendly chess game that supports both player-versus-player (PVP) and player-versus-engine (PVE) modes. The application is a standalone desktop program built using the Java Swing library for its graphical user interface. It successfully implements all standard chess rules, including special moves, and features an intelligent AI opponent.

2. Key Features

The application boasts a rich set of features that provide a complete and engaging chess-playing experience:

- **Full Chess Logic:** The game correctly implements all the rules of chess, including:
 - Standard piece movement (Pawn, Rook, Knight, Bishop, Queen, King).
 - **Special Moves:**
 - **Castling:** Both kingside and queenside castling are supported.
 - **En Passant:** The special pawn capture rule is correctly implemented.
 - **Pawn Promotion:** Pawns are automatically promoted to a Queen upon reaching the final rank.
 - **Game State Detection:** The engine accurately detects conditions of check, checkmate, and stalemate.
- **Game Modes:**
 - **Player vs. Player (PVP):** Two human players can play against each other on the same computer.
 - **Player vs. Environment (PVE):** A human player can compete against an integrated AI opponent.
- **Graphical User Interface (GUI):** The application features a clean and intuitive graphical interface built with Java Swing. This includes:
 - A visually clear and interactive chessboard.
 - Highlighting for the selected piece and its valid moves.
 - A status bar that displays whose turn it is and if a player is in check.
 - Timers for both white and black players to manage game duration.
 - A "Game Over" screen with options to play again.
 - Panels to display pieces captured by each player.
- **Improved AI Bot:** The AI opponent uses a 1-ply (one move ahead) lookahead algorithm with a material evaluation function. This allows the bot to make strategic decisions by assessing the board state after each potential

move, providing a more challenging experience than a simple greedy algorithm.

- **Captured Pieces and Score:** The interface includes a panel that displays the pieces each player has captured. It also shows a material advantage score, giving players a quick insight into who is leading.

3. Software Architecture

The application is well-structured and organized into several key classes, each with a distinct responsibility. All classes are conveniently packaged within a single `Chess.java` file.

- **Chess (Main Class):** This class extends `JFrame` and serves as the main entry point and primary container for the application. It is responsible for:
 - Setting up the main game window and all GUI components.
 - Managing the game state, including timers and game modes.
 - Handling the "Game Over" and "Play Again" logic.
- **Board:** This class extends `JPanel` and is responsible for all logic and rendering directly related to the chessboard itself. Its duties include:
 - Drawing the board, pieces, and move highlights.
 - Handling mouse input for piece selection and movement.
 - Maintaining the internal 2D array representing the board state.
 - Validating moves and detecting check/checkmate/stalemate conditions.
- **Piece (Abstract Class):** This is the base class for all chess pieces. It defines common attributes and methods like color, position, and value. Concrete piece classes (`King`, `Queen`, `Rook`, `Bishop`, `Knight`, `Pawn`) extend this class and implement their specific movement logic.
- **Move:** A simple data structure class used to represent a move from a starting square to an ending square. It also stores information about special move types like castling or en passant.
- **Bot:** This class contains the logic for the AI opponent. It analyzes the board and uses its evaluation function to determine the best move to make.
- **CapturedPiecesPanel:** A custom `JPanel` component responsible for displaying the captured pieces for each side.

4. Conclusion

The Java Chess project successfully meets all its stated objectives. It is a robust and feature-complete application that provides an enjoyable chess-playing experience for both casual and intermediate players. The implementation demonstrates a strong understanding of object-oriented programming principles, GUI design with Java Swing, and game development logic. The inclusion of a strategic AI opponent adds significant value and replayability to the application.